

Tổng Hợp Kiến Thức

1. Định tuyến dựa trên hệ thống tệp (File-System Routing)

- **Khái niệm:** Mỗi thư mục trong `app` đại diện cho một **segment tuyến đường**. File `page.js` định nghĩa giao diện (UI) cho tuyến đường đó.
- **Ví dụ:**
 - `app/page.js` → Tuyến đường `/`
 - `app/about/page.js` → Tuyến đường `/about`
 - `app/dashboard/settings/page.js` → Tuyến đường `/dashboard/settings`
- **File chính:** `page.js` (mặc định là Server Component, render trên server).

2. Root Layout

- **File:** `app/layout.js`
- **Mục đích:** Xác định layout chung cho **tất cả các trang**. **Phải** chứa thẻ `<html>` và `<body>`.
- **Props:** Nhận `children` (nội dung của các trang hoặc layout con).
- **Ví dụ:**

```
// app/layout.js
import './globals.css';
export default function RootLayout({ children }) {
  return (
    <html lang="vi">
      <body>
        <header>LetDiv</header>
        <main>{children}</main>
        <footer>©2025 LetDiv</footer>
      </body>
    </html>
  );
}
```

- **Lưu ý:**
 - Sử dụng object `metadata` để quản lý SEO (`title` , `description`).
 - Root Layout không re-render khi chuyển trang.

3. CSS Styling

- **CSS Toàn cục:** `app/globals.css` cho các style áp dụng toàn ứng dụng.
- **CSS Modules:** Style cục bộ, chỉ áp dụng cho component/layout cụ thể.
 - Ví dụ: `app/about/about.module.css` kết hợp với `app/about/page.js`.

```
/* app/about/about.module.css */
.container { padding: 20px; background-color: #f0f0f0; }
.title { color: navy; }
```

```
// app/about/page.js
import styles from './about.module.css';
export default function AboutPage() {
  return (
    <div className={styles.container}>
      <h1 className={styles.title}>Giới thiệu</h1>
      <p>Đây là trang giới thiệu.</p>
    </div>
  );
}
```

```
</div>
);
}
```

4. Thuật ngữ điều hướng

- **URL:** Địa chỉ web đầy đủ (VD: `https://example.com/blog/post`).
- **Path (Đường dẫn):** Phần sau domain (VD: `/blog/post`).
- **Route (Tuyến đường):** Đường dẫn trong ứng dụng, được định nghĩa bởi cấu trúc thư mục và `page.js` (VD: `/about`).
- **Segment (Phân đoạn):** Mỗi phần của path, ngăn cách bởi `/` (VD: `dashboard` , `settings` trong `/dashboard/settings`).

5. Liên kết với `<Link>`

- **Component:** `next/link` để điều hướng phía client (không tải lại toàn trang).
- **Ví dụ:**

```
import Link from 'next/link';
<nav>
  <Link href="/">Trang chủ</Link>
  <Link href="/about">Giới thiệu</Link>
  <Link href="/contact">Liên hệ</Link>
</nav>
```

- **Lưu ý:** Chỉ fetch và render nội dung trong `children` .

6. Điều hướng chương trình với `useRouter`

- **Hook:** `useRouter` từ `next/navigation` (chỉ dùng trong Client Component, cần `'use client'`).
- **Phương thức chính:**
 - `router.push('/path')` : Chuyển đến tuyến đường mới, thêm vào lịch sử trình duyệt.
 - `router.replace('/path')` : Chuyển đến tuyến đường mới, không lưu lịch sử.
 - `router.back()` : Quay lại trang trước.
 - `router.forward()` : Chuyển đến trang tiếp theo.
 - `router.refresh()` : Làm mới tuyến đường hiện tại.
- **Ví dụ:**

```
// app/login/page.js
'use client';
import { useRouter } from 'next/navigation';
export default function LoginPage() {
  const router = useRouter();
  return (
    <button onClick={() => router.push('/dashboard')}>
      Đi đến Dashboard
    </button>
  );
}
```

7. Tuyến đường lồng nhau (Nested Routes)

- **Khái niệm:** Tuyến đường trong thư mục con (VD: `app/dashboard/settings/page.js` → `/dashboard/settings`).
- **Ví dụ:**

```
// app/dashboard/page.js
export default function DashboardPage() {
  return <h1>Dashboard</h1>;
}
// app/dashboard/settings/page.js
export default function SettingsPage() {
  return <h2>Cài đặt</h2>;
}
```

8. Layout lồng nhau (Nested Layouts)

- **File:** `layout.js` trong thư mục con áp dụng cho tất cả tuyến đường con của segment đó.
- **Ví dụ:**

```
// app/dashboard/layout.js
import Link from 'next/link';
export default function DashboardLayout({ children }) {
  return (
    <section>
      <nav>
        <Link href="/dashboard">Tổng quan</Link>
        <Link href="/dashboard/settings">Cài đặt</Link>
      </nav>
      <div>{children}</div>
    </section>
  );
}
```

- **Lưu ý:** Layout lồng nhau không re-render khi chuyển giữa các tuyến đường con.

9. Các file UI đặc biệt

- **`loading.js` :**
 - Hiển thị UI khi tuyến đường đang tải (dùng React Suspense).
 - Ví dụ: `app/dashboard/loading.js`

```
export default function DashboardLoading() {
  return <p>Đang tải...</p>;
}
```

- **`error.js` :**
 - Client Component (`'use client'`) để xử lý lỗi trong segment.
 - Props: `error` (đối tượng lỗi), `reset` (hàm thử lại).
 - Ví dụ: `app/dashboard/error.js`

```
'use client';
export default function DashboardError({ error, reset }) {
  return (
    <div>
      <h2>Lỗi: {error.message}</h2>
      <button onClick={reset}>Thử lại</button>
    </div>
  );
}
```

- **`not-found.js` :**

- Hiển thị khi tuyến đường không tồn tại hoặc gọi `notFound()` .
- Ví dụ: `app/not-found.js`

```
import Link from 'next/link';
export default function NotFound() {
  return (
    <div>
      <h1>404 - Không tìm thấy trang</h1>
      <Link href="/">Về trang chủ</Link>
    </div>
  );
}
```

- `template.js` :
 - Tương tự `layout.js` nhưng tạo instance mới mỗi khi điều hướng (ít dùng, không cần chi tiết).

10. Route Groups

- **Khái niệm:** Tổ chức tuyến đường mà không ảnh hưởng URL, dùng thư mục `(group_name)` .
- **Ví dụ:**
 - `app/(marketing)/about/page.js` → `/about`
 - `app/(auth)/login/page.js` → `/login`
- **Mục đích:** Chia ứng dụng thành các nhóm logic, mỗi nhóm có `layout.js` riêng.
- **Lưu ý:** Kiểm tra URL không bị ảnh hưởng bởi tên nhóm.